Simplifying Scientific Text

Graduation Project 1
prepared to obtain the degree of
B.Sc. in the Department of Artificial Intelligence Engineering and Data Science

Author:

Raneem Rabih

Supervised by:

Dr. Riad Sonbol
Eng. Wissam Al-Sahli

Damascus
2026

Abstract

Scientific and medical texts are often difficult for non-specialist readers to understand because they contain technical terminology, long sentence structures, and dense statistical or medical information that may require prior domain knowledge. Therefore, scientific text simplification is an important Natural Language Processing task that aims to make scientific knowledge more accessible and understandable. However, this task is not limited to rewriting text in simpler language. It also requires preserving the original scientific meaning, avoiding the deletion of important information, preventing unsupported additions, and carefully handling medical terminology, uncertainty, and numerical details.

This project proposes a sentence-level scientific and medical text simplification system based on a controlled and interpretable large-language-model pipeline. The system does not fine-tune the model or update its weights. Instead, it uses the model in inference mode through several organized stages. The process starts with text normalization and terminology extraction, followed by the construction of terminology-based context to help the model handle difficult scientific and medical terms more safely. The system then applies a semantic planning stage that returns a structured JSON output, allowing the system to analyze sentence properties such as terminology, structural complexity, and secondary statistical details. Based on this analysis, a decision layer selects the appropriate simplification strategy, such as rephrasing, splitting and simplifying, keeping the sentence unchanged, or omitting secondary information when appropriate. The added value of the proposed system is that it does not rely on a single direct prompt. Instead, it separates analysis from execution, supports traceable intermediate decisions, and applies terminology-aware substitution before the final simplification strategy when needed.

The proposed system was evaluated on two samples of 500 sentences: one from the validation set and one from the test set. The system achieved a SARI score of 38.98 on the validation sample and 38.84 on the test sample, indicating relatively stable performance across the two samples. The average FKGL score decreased from 12.64 to 11.33 on the validation sample and from 13.39 to 11.58 on the test sample, showing improved readability after simplification. In addition, the compression ratio was 0.868 on the validation sample and 0.871 on the test sample, which indicates that the system generally produced shorter and more concise outputs. These results suggest that the proposed system can improve the readability and conciseness of scientific and medical sentences while maintaining stable and reasonable simplification performance according to the automatic evaluation metrics.

Keywords: Text Simplification, Scientific Text, Medical Text, Large Language Models, Prompt Engineering, Terminology Support, SARI, FKGL.

Table of contents

Table index

Index of equations

Index of figures

رمز / اختصار	المقابل بالإنكليزية	المصطلح بالعربية
AI	Artificial Intelligence	الذكاء الاصطناعي
NLP	Natural Language Processing	معالجة اللغة الطبيعية
LLM	Large Language Model	نموذج لغوي كبير
LLMs	Large Language Models	النماذج اللغوية الكبيرة
Transformer	Transformer Architecture	بنية المحوّل
Self-Attention	Self-Attention Mechanism	آلية الانتباه الذاتي
Tokenizer	Tokenizer	مقطّع النص إلى رموز
Embeddings	Token Embeddings	التمثيلات المتجهية للرموز
Inference	Inference Mode	وضع الاستدلال
Fine-tuning	Fine-tuning	الضبط الدقيق / إعادة التدريب التخصصي
Prompt	Prompt	التعليمة المدخلة للنموذج
Prompt Engineering	Prompt Engineering	هندسة التعليمات
Qwen2.5-14B-Instruct	Qwen2.5 14B Instruction-Tuned Model	نموذج Qwen2.5 بحجم 14 مليار معامل والمهيأ للتعليمات
GPT	Generative Pretrained Transformer	المحوّل التوليدي المدرب مسبقًا
BERT	Bidirectional Encoder Representations from Transformers	تمثيلات BERT ثنائية الاتجاه
BART	Bidirectional and Auto-Regressive Transformers	نموذج BART
T5	Text-to-Text Transfer Transformer	نموذج T5
Text Simplification	Text Simplification	تبسيط النصوص
Scientific Text Simplification	Scientific Text Simplification	تبسيط النصوص العلمية
Medical Text Simplification	Medical Text Simplification	تبسيط النصوص الطبية

Sentence-level Simplification	Sentence-level Text Simplification	تبسيط النصوص على مستوى الجملة
Document-level Simplification	Document-level Text Simplification	تبسيط النصوص على مستوى الوثيقة
Meaning Preservation	Meaning Preservation	الحفاظ على المعنى
Readability	Readability	قابلية القراءة
Hallucination	Hallucination	الهلوسة
Faithfulness	Faithfulness	الالتزام بالمعنى الأصلي
Terminology Support	Terminology Support	دعم المصطلحات
Glossary	Glossary	قاموس مصطلحات
MedSimplify	MedSimplify Glossary	قاموس MedSimplify
Term Extraction	Term Extraction	استخراج المصطلحات
Terminology Context	Terminology Context	السياق المصطلحي
RAG	Retrieval-Augmented Generation	التوليد المعزز بالاسترجاع
Pipeline	Processing Pipeline	خط الأنابيب / مسار المعالجة
Semantic Planning	Semantic Planning	التخطيط الدلالي
Planner	Planner	مخطّط النظام
Executor	Executor	منفّذ النظام
Strategy Selection	Strategy Selection	اختيار الاستراتيجية
JSON	JavaScript Object Notation	صيغة تبادل بيانات منظمة
CSV	Comma-Separated Values	ملف قيم مفصولة بفواصل
Checkpointing	Checkpointing	الحفظ المرحلي
KEEP_AS_IS	Keep As Is Strategy	استراتيجية الإبقاء كما هو
REPHRASE	Rephrase Strategy	استراتيجية إعادة الصياغة
SPLIT_AND_SIMPLIFY	Split and Simplify Strategy	استراتيجية التقسيم والتبسيط

DELETE_OR_OMIT	Delete or Omit Strategy	استراتيجية الحذف أو الإهمال
SUBSTITUTE	Terminology Substitution Strategy	استراتيجية الاستبدال المصطلحي
Cochrane-auto	Cochrane-auto Dataset	مجموعة بيانات Cochrane-auto
Validation Set	Validation Set	مجموعة التحقق
Test Set	Test Set	مجموعة الاختبار
pair_id	Pair Identifier	معرّف زوج النص المعقد والمبسط
para_id	Paragraph Identifier	معرّف الفقرة
sent_id	Sentence Identifier	معرّف الجملة
run_id	Run Identifier	معرّف التشغيل
SARI	System output Against References and Input sentence	مقياس SARI لتقييم التبسيط
BLEU	Bilingual Evaluation Understudy	مقياس BLEU للتشابه اللفظي
FKGL	Flesch-Kincaid Grade Level	مستوى القراءة حسب فليش-كينكيد
FKGL Delta	FKGL Difference	فرق مستوى القراءة
Compression Ratio	Compression Ratio	نسبة الضغط
Gradio	Gradio Interface Framework	إطار Gradio لبناء الواجهة
Codabench	Codabench Evaluation Platform	منصة Codabench للتقييم
CLEF	Conference and Labs of the Evaluation Forum	مؤتمر ومختبرات منتدى التقييم
SimpleText	SimpleText Track	مسار SimpleText
Task 1.1	Sentence-level Scientific Text Simplification Task	مهمة تبسيط النصوص العلمية على مستوى الجملة

1 Chapter One

1.1 Introduction

Natural Language Processing has developed significantly with the emergence of large language models, which are now able to understand, rewrite, and generate text in a flexible and coherent manner. Text simplification is one of the important applications in this field, as it aims to transform complex text into a clearer and easier form while preserving the original meaning.

The need for text simplification becomes more important in scientific and medical domains, where texts often contain specialized terminology, long sentence structures, and detailed information that may be difficult for non-specialist readers to understand. This project focuses on building an intelligent system for simplifying scientific and medical sentences using large language models, with the goal of making information more accessible without changing the essential meaning.

1.2 Problem of Project

The scientific problem addressed in this project is the difficulty of understanding scientific and medical texts by non-specialist readers. Such texts often include complex terminology, long grammatical structures, and domain-specific details. Although large language models can rewrite text, using them directly may sometimes lead to removing important information, changing the meaning, or adding unsupported content.

Therefore, there is a need for an intelligent simplification system that does not process all sentences in the same way. Instead, the system should determine the suitable simplification operation for each sentence, such as rephrasing, splitting, omitting unnecessary details, substituting difficult terms, or keeping the sentence unchanged, and then generate a simpler version while preserving the core scientific meaning.

1.3 Project Objectives

The main objective of this research is to build an intelligent system that helps make scientific and medical sentences easier to understand for non-specialist readers while preserving the original meaning of the source text. The research aims to reduce the gap between complex scientific writing and general readers by improving the clarity and readability of specialized texts, especially those that contain difficult terminology, long sentence structures, and dense information. It also seeks to support accurate access to scientific and medical knowledge by simplifying the language without adding unsupported information, removing essential content, or changing the intended meaning.

1.4 Project Contribution

The main contribution of this project is the development of a structured text simplification system that combines operation selection with large-language-model-based generation. Instead of applying a single fixed rewriting prompt to all sentences, the proposed system first analyzes the sentence and determines the appropriate type of simplification, then generates the simplified output according to that decision.

The project also contributes by combining several components within one pipeline: large language models, operation-aware planning, and optional medical term support. This design allows the system to handle different types of scientific sentences, including long sentences, terminology-heavy sentences, and sentences that contain secondary details. In addition, the project provides an experimental evaluation of the proposed approach using both quantitative metrics and qualitative analysis.

1.5 Beneficiaries of the Project

Several groups can benefit from this project, including:

- Non-specialist readers who need to understand scientific and medical information more easily.
- Patients and caregivers who may benefit from clearer medical explanations.
- Students and early-stage projecters who need support in understanding scientific summaries and abstracts.
- Educational institutions that aim to prepare simplified scientific learning materials.
- Health and educational platforms that provide accessible content for the general public.
- Projecters in Natural Language Processing who study text simplification and large language model applications.

1.6 Thesis Organization

This thesis is organized into six chapters. Chapter One introduces the project topic, the scientific problem, the project objectives, the contribution, and the expected beneficiaries. Chapter Two presents the theoretical background related to Natural Language Processing, text simplification, large language models, and evaluation metrics.

Chapter Three reviews related work and previous studies in scientific and medical text simplification, with a discussion of the project gap addressed by this project. Chapter Four explains the proposed methodology, including the system architecture, dataset, preprocessing steps, operation selection, generation process, and experimental setup. Chapter Five presents the evaluation results and compares the proposed strategies. Finally, Chapter Six summarizes the main conclusions, discusses the limitations, and suggests future improvements.

2 Chapter Two: Theoretical Background

This chapter presents the theoretical and computational concepts required to understand the proposed project.

2.1 Natural Language Processing

Natural Language Processing (NLP) is a branch of artificial intelligence concerned with enabling computers to process, analyze, understand, and generate human language. NLP is used in many applications such as machine translation, text classification, information extraction, summarization, question answering, and text simplification.

In this project, NLP is the main computational field because the system receives textual input and produces a modified textual output. The goal is not only to process the sentence structurally, but also to generate a simpler version that remains faithful to the original meaning.

2.2 Text Simplification

Text simplification is the task of rewriting a complex text into a clearer and more readable form while preserving its essential meaning. The objective is to reduce linguistic complexity without changing the core information expressed in the original text.

Simplification may involve several linguistic changes, such as replacing difficult words with simpler alternatives, rewriting complex expressions, shortening long constructions, or restructuring a sentence to make it easier to read. In some cases, the best output may be very close to the original sentence if the sentence is already understandable.

In this project, text simplification is addressed at the sentence level. Each input sentence is treated as an independent unit that should be simplified according to its content and linguistic structure.

2.3 Neural Language Models

Neural language models are computational models that learn patterns of language from large text datasets. Instead of relying only on manually written linguistic rules, these models learn statistical and semantic relationships between words and phrases through training.

A language model estimates the likelihood of word sequences and uses this knowledge to generate or transform text. Modern neural language models represent words or tokens as numerical vectors, allowing the model to capture relationships between linguistic units in a continuous mathematical space.

These models form the basis of many modern NLP systems because they can learn contextual information and produce fluent text. However, their output still depends strongly on the training data, model architecture, and instructions provided during inference.

2.4 Transformer Architecture

The Transformer is one of the most important architectures used in modern NLP. It was designed to process sequential data, especially natural language, using an attention mechanism instead of relying only on recurrent processing.

The main idea of the Transformer is self-attention. Self-attention allows the model to focus on different parts of the input sequence when representing or generating each token. This enables the model to capture long-range dependencies and contextual relationships more effectively than many earlier architectures.

A Transformer model usually includes several core components, such as token embeddings, positional information, attention layers, and feed-forward neural networks. These components allow the model to transform an input sequence into contextual representations that can be used for understanding or generation tasks.

2.5 Large Language Models

Large Language Models (LLMs) are advanced neural language models trained on very large text corpora. They are capable of generating coherent text, answering questions, summarizing content, following instructions, and performing several NLP tasks without task-specific training in some cases.

LLMs are useful for text simplification because they can rewrite complex sentences in simpler language. However, their use must be controlled carefully. A model may sometimes simplify too aggressively, remove important information, or generate unsupported content. Therefore, the quality of the output depends on the model, the prompt, the constraints, and the evaluation process.

Examples of large language model families that may be considered in this project include LLaMA, Qwen, Gemini, Gemma, and other instruction-tuned models. The final selection depends on experimental results, computational feasibility, cost, and output quality.

2.6 Prompt Engineering

Prompt engineering is the process of designing clear and effective instructions for a language model. Since large language models respond according to the input prompt, the structure and wording of the prompt have a direct effect on the generated output.

In text simplification, prompt engineering is important because the model must be guided to produce simpler text while preserving the original meaning. A good prompt should define the task, specify the expected behavior, and include constraints such as avoiding unsupported information, preserving important details, and using clear language.

Prompt design can also help improve consistency, especially when the system requires outputs in a specific format or when different types of sentence transformations are needed.

2.7 Terminology Support in Scientific Text Simplification

Scientific texts often contain specialized terms that may be difficult to simplify correctly. To support the simplification process, a system may use a terminology-support component, such as a glossary of domain-specific terms and short definitions.

In this project, terminology support is considered as an auxiliary component rather than a full retrieval-based question answering system. Its role is to provide limited contextual hints about difficult scientific or medical terms when they appear in the input sentence. These hints can help the language model understand the meaning of specialized terms and choose clearer wording.

This component should not be used to add new information that is not present in the original sentence. Instead, it should support faithful simplification by helping the model preserve meaning while making the sentence easier to understand.

2.8 Structure of the Qwen2.5-14B-Instruct Model

Qwen2.5-14B-Instruct is a transformer-based large language model. More specifically, it follows a decoder-only Transformer architecture, which is the common architecture used in many modern generative language models. This means that the model is designed to receive a sequence of input tokens and generate the next tokens autoregressively, one token at a time.
The general structure of the model can be described as a stack of Transformer decoder layers. The input sentence is first converted into tokens using the model tokenizer. These tokens are then represented as numerical vectors called embeddings. After that, the embeddings pass through several Transformer layers, where the model captures relationships between words and uses the context of the sentence to produce a suitable output.
Each Transformer layer contains an attention mechanism and a feed-forward network. The attention mechanism allows the model to focus on different parts of the input sentence when generating or analyzing text. This is important for text simplification because the model must understand the relationship between medical terms, modifiers, numerical details, and the main meaning of the sentence.

Qwen2.5 uses several architectural components commonly found in recent large language models. These include rotary positional embeddings, which help the model understand the order of tokens; grouped-query attention, which improves the efficiency of attention computation; and feed-forward layers with modern activation functions such as SwiGLU. These components help the model process long and complex text more efficiently while maintaining strong generation ability. Qwen2.5 models are described as dense decoder-only language models, and the model family includes several sizes, including the 14B version used in this project.

The "14B" in the model name indicates that the model contains approximately fourteen billion parameters. These parameters are the learned numerical values that store linguistic and contextual patterns acquired during pretraining. The relatively large number of parameters allows the model to handle complex language tasks such as instruction following, sentence analysis, planning, and simplification generation.

The "Instruct" version means that the model has been further adapted to follow user instructions. This is important for the proposed system because the model is required to follow controlled prompts, produce structured plans, and generate simplified sentences according to specific strategies. In this project, the model is used in two main roles: first, to analyze the sentence and generate a structured simplification plan, and second, to generate the final simplified sentence according to the selected strategy.

In summary, the structure of Qwen2.5-14B-Instruct can be viewed as a decoder-only Transformer model composed of tokenization, embedding representation, stacked self-attention layers, feed-forward layers, and an output generation layer. This structure makes it suitable for the proposed simplification pipeline because it can understand sentence context, follow instructions, produce structured outputs, and generate fluent simplified text.

3 Chapter Three: The CLEF SimpleText Shared Task

This project is developed within the context of the CLEF SimpleText shared task, which focuses on making scientific text easier to understand using Natural Language Processing methods. The task provides a common evaluation setting where systems can be compared using shared data and evaluation metrics. SimpleText includes simplification at both the sentence level and the document level; however, this project focuses on Task 1.1: sentence-level scientific text simplification, where the system receives a complex scientific or medical sentence and generates a simpler version while preserving the original meaning. The task uses data derived from Cochrane-auto, which contains aligned scientific abstracts and plain-language summaries. In this project, the sentence-level part is considered at the current stage, while the dataset structure, preprocessing steps, and data splits will be described in detail in the methodology chapter.

4 Chapter Four: Related Work

4.1 Introduction

This chapter reviews the main research papers presented in the **SimpleText 2025** lab. The reviewed works are organized according to the technical methodologies they used for scientific and medical text simplification. These methodologies include zero-shot prompting and prompt engineering, fine-tuning and supervised learning, hallucination detection, plan-driven simplification, and lexical or terminology-supported simplification.

The purpose of this review is to clarify the main research directions in the field, highlight the strengths and limitations of each methodology, and position the current project within these related works. In particular, the current project is related to approaches that combine operation-aware planning, large language models, and limited terminology support for controlled sentence-level simplification.

Zero-Shot Prompting and Prompt Engineering Approaches

A major group of SimpleText 2025 systems relied on zero-shot prompting and prompt engineering. These approaches attempt to use the existing capabilities of large language models without additional task-specific training. Their performance depends mainly on the quality of the prompt, the selected model, and the constraints imposed during generation.

The LIA team from Avignon used human-inspired simplification guidelines derived from Cochrane recommendations to guide models such as Mistral and LLaMA in document simplification. Their best system, referred to as Guidelines-all, achieved a SARI score of 44.93, indicating strong performance among prompt-based systems. This approach highlights the importance of providing the model with explicit simplification principles rather than relying on a general rewriting instruction.

The SINAI team used GPT-4.1 for medical text simplification, with particular attention to explaining difficult terms between parentheses. Their system achieved a SARI score of 41.25 in Task 1.1 and 43.63 in Task 1.2. This work shows that prompt engineering can be effective when the model is instructed to handle terminology explicitly.

The UBOnlp team explored the raw simplification ability of GPT-4o using a simple prompt. Their method focused on reducing content and generating simpler text. The system achieved a SARI score of 42.20 at the sentence level and 43.37 at the document level. Although the approach was relatively simple, the results suggest that strong general-purpose LLMs can perform competitively even with limited task-specific engineering.

The Fujitsu Lenguaje-Claro team compared zero-shot prompting with three-shot prompting, where the examples were designed to represent specific linguistic phenomena. Their GPT-3.5-Turbo three-shot configuration achieved a SARI score of 38.84. This result suggests that fewshot prompting may improve control in some cases, although its effectiveness depends on the quality and representativeness of the selected examples.

Fine-Tuning and Supervised Learning Approaches

Another group of systems relied on fine-tuning or supervised learning. These approaches train or adapt models directly on simplification datasets, such as Cochrane-auto, in order to improve task-specific performance.

The UM_FHS team compared prompt-based methods with fine-tuned versions of GPT-4.1 mini and GPT-4.1 nano. Their gpt-4.1-mini without context achieved a SARI score of 42.13. This result shows that smaller instruction-tuned models can achieve competitive performance when appropriately adapted or prompted.

The NITK SCaLAR team fine-tuned GPT-2 Medium for direct simplification and also proposed a hybrid system combining BioBERT and BioBART. Their GPT-2 Medium system achieved a SARI score of 39.77. This work reflects the use of supervised neural models for simplification, although its performance remained below the strongest prompt-engineering systems.

The AIIRLab team used Unsloth to fine-tune quantized versions of Mistral and LLaMA models. Their Mistral-7B system achieved a SARI score of 36.08 at the sentence level and 42.40 at the document level. This approach is relevant because it shows how quantized models can be adapted for simplification under computational constraints.

The DUTH team applied instruction tuning to models such as FLAN-T5-Large and BART-SAMSum. Their FLAN-T5-Large model achieved a SARI score of 35.35. Although the result was lower than several LLM-based systems, this approach remains important because encoder-decoder models such as T5 are naturally suitable for sequence-to-sequence tasks like simplification.

In general, fine-tuning approaches offer better task specialization, but their performance depends strongly on the training data, model size, and available computational resources. In comparison, prompt-based LLM systems achieved higher scores in several cases without requiring full model training.

Hallucination Detection Approaches

Hallucination detection was mainly associated with the second task of SimpleText 2025, which focused on identifying unsupported or inaccurate information in simplified texts. This direction is important because simplification systems may produce fluent text while changing or adding information that is not supported by the source.

The SINAI team combined rule-based pattern detection, such as identifying single-word sentences, with LLaMA-3.1. Their best system achieved an F1-score of 0.976 for source-based hallucination detection. This result indicates that combining simple rules with large language models can be highly effective for detecting unsupported content.

The DS@GT team developed a hybrid hallucination detection system that combined BERT classifiers, Natural Language Inference models, and LLMs. Their system achieved an F1-score of 0.95 when the source was available. This approach shows the value of combining classification and inference-based methods to verify whether a simplified output remains faithful to its source.

The DUTH team used tree-based classifiers, particularly Extra Trees, to analyze lexical and semantic features. Their system achieved an F1-score of 0.950 for hallucination detection. This result shows that traditional machine learning methods can still be useful when supported by carefully designed features.

Plan-Driven and Structural Simplification Approaches

Plan-driven and structural approaches attempt to decompose the simplification process into explicit steps. Instead of applying a single rewriting instruction to all sentences, these systems identify a simplification strategy or structural transformation before generating the final output.

The DS@GT team, which achieved the highest sentence-level performance in Task 1, used a plan-guided system in which the model selected a strategy such as deletion or splitting before generating the simplified text. Their system achieved a SARI score of 42.98. This result is especially relevant to the current project because it supports the idea that operation-aware simplification can improve control over sentence transformations.

The PICT team proposed the S-3 Pipeline, which combined structural AMR representations with semantic simplification using T5 and lexical simplification using BERT-Masking. Their system achieved a SARI score of 17.77. The team interpreted the low score in the medical context as potentially reflecting stronger meaning preservation. This approach is important because it shows an attempt to use deeper semantic structure rather than relying only on surface-level rewriting.

The UvA team from Amsterdam applied a Plan-guided BART system that identifies the required action for each sentence before processing it. Their system achieved a SARI score of 38.5 in the sentence-level task. This work further supports the relevance of planning in text simplification, especially when different sentences require different transformation operations.

Retrieval-Augmented and Lexical Simplification Approaches

Some systems focused on lexical simplification and external terminology support. These approaches are particularly important in scientific and medical text simplification because difficult terminology is one of the main barriers for non-specialist readers.

The LIS team introduced a notable contribution through the MedSimplify glossary, which contains approximately 3000 definitions. They used this resource within a retrieval-augmented architecture. Their Mistral_DASP_0 system achieved a SARI score of 43.51. This result indicates that terminology support can improve simplification by helping the model handle medical or scientific expressions more effectively.

The THM team focused on Complex Word Identification. Their system identified complex terms and marked them with square brackets so that the model would simplify those terms specifically. Using gpt-4.1-nano, the system achieved a SARI score of 39.57. This approach shows that explicitly identifying difficult terms can guide the model toward more targeted simplification.

Comparative Analysis of Reviewed Approaches

The reviewed systems show that there is no single dominant method for scientific text simplification. Prompt engineering approaches achieved strong results, especially when combined with clear simplification guidelines. Fine-tuning approaches provided task-specific adaptation, but did not always outperform strong LLM prompting. Plan-driven systems showed the importance of selecting a suitable simplification strategy before generation, while lexical and retrieval-supported systems demonstrated the value of terminology handling.

4.2 Modern Techniques and Tools in the Field

Recent work in scientific and medical text simplification relies on several modern Natural Language Processing techniques. The most important foundation is the Transformer architecture, which allows language models to capture contextual relationships between words through self-attention. This architecture has enabled the development of large language models that can understand instructions and generate fluent rewritten text.

Large Language Models are widely used in recent simplification systems because they can perform rewriting, paraphrasing, sentence splitting, and terminology explanation without requiring full task-specific training. However, their outputs must be controlled carefully because they may remove important information, over-simplify the sentence, or generate unsupported content.

Prompt engineering is another important technique in this field. By designing clear prompts, the system can guide the model to preserve the original meaning, avoid hallucination, use simpler vocabulary, and follow a specific simplification operation. This is especially important in scientific and medical domains, where factual accuracy is critical.

Retrieval-Augmented Generation and terminology-support methods are also relevant. Instead of relying only on the model's internal knowledge, these methods provide external definitions or glossary entries for difficult terms. In this project, terminology support is used in a limited and controlled way to help the model understand specialized medical terms without adding new information that is not present in the source sentence.

Finally, automatic evaluation metrics are commonly used to compare simplification systems. SARI is one of the main metrics used in text simplification because it evaluates how well a system adds, deletes, and keeps words compared with reference simplifications. BLEU can measure overlap with references, while readability metrics such as FKGL can estimate whether the generated sentence became easier to read.

4.3 Comparative analysis of systems

Table 4.1: Comparative Analysis of Reviewed Systems

Team / System	Main Methodology	Models Used	Main Reported Result	Strengths	Limitations
UM-FHS	Prompt engineering and fine-tuning with simplification guidelines	GPT-4.1, GPT-4.1-mini, GPT-4.1-nano	SARI 43.34 in Task 1.1; SARI 43.83 in Task 1.2	Strong prompt design; clear simplification guidelines; competitive results	Closed and costly models; fine-tuning results were not always stable; no explicit planning pipeline
LIA	Cochrane-guideline-based prompting for document simplification	LLaMA, Mistral, Gemma	SARI 44.93 in Task 1.2	Strong use of medical simplification guidelines; best document-level performance	Focused on Task 1.2; large models; no post-generation verification
DS@GT	Plan-driven simplification before generation	LLaMA 3.3 70B	SARI 42.98 in Task 1.1	Planning improves control and interpretability; close to current	Requires multiple model calls; possible aggressive deletion; no explicit

Team / System	Main Methodology	Models Used	Main Reported Result	Strengths	Limitations
				project direction	verification stage
SINAI	Zero-shot medical prompting and hallucination detection	GPT-4.1, LLaMA-3.1	SARI 41.25–41.82 in Task 1.1; F1 0.976 in hallucination detection	Strong zero-shot performance; explicit handling of terms; strong hallucination detection	Depends on closed large models; zero-shot outputs may be unstable on complex cases
LIS	Terminology-supported simplification using MedSimplify glossary	Mistral 7B, Gemma, Med42	SARI 43.51 in Task 1.2	Domain terminology support; no fine-tuning required; relevant to medical simplification	Exact matching only; limited glossary coverage; mainly document-level
UvA	Fine-tuned BART and jargon-aware prompting	BART, LLaMA 3.1 8B, MedReadMe classifier	SARI 42.31 in Task 1.1	Effective use of Cochrane-auto fine-tuning; jargon-aware prompting	No explicit planning stage; no verification after generation
THM	Complex word identification and targeted lexical simplification	GPT-4.1-nano, Gemini models	SARI 41.32 in Task 1.1	Focused handling of difficult terms; economical model option	Closed models; mainly lexical focus; no full structural pipeline
DUTH	Encoder-decoder simplification and hallucination classification	FLAN-T5, BART-SAMSum, Extra Trees	SARI 38.73 in Task 1.1; F1 0.948 in hallucination detection	Practical smaller models; covers simplification and hallucination detection	Lower SARI than LLM-based systems; no medical guidelines or planning

Team / System	Main Methodology	Models Used	Main Reported Result	Strengths	Limitations
AIIRLab	Quantized LLMs with cross-encoder verification	Mistral 7B, LLaMA 3.1/3.2, BERT-based cross-encoders	SARI 42.44 with base Mistral; SARI 36.08 after fine-tuning	Open-source medium-size models; verification-oriented design	Cross-encoder adds latency; fine-tuning did not always improve performance
PICT S-3 Pipeline	Hybrid lexical and semantic simplification	SciBERT, AMRlib, PEGASUS, BERT, T5	SARI 40.15 in Task 1.1; SARI 40.29 in Task 1.2	Highly structured and interpretable pipeline; lexical and semantic processing	Complex architecture; lower SARI than top LLM systems; no explicit planning
NITK SCaLAR	Biomedical model pipeline and GPT-2 fine-tuning	BioBERT, BioBART, GPT-2 Medium, SciSpacy	SARI 33.95–39.77 in Task 1.1	Uses biomedical representations; lower computational cost	Lower performance; older models; no planning or verification
UBOnlp	Direct prompting with a general-purpose LLM	GPT-4o	SARI 40.74–42.20 in Task 1.1; SARI 41.56–43.37 in Task 1.2	Strong results with a simple setup	Closed and costly model; limited methodological detail; no structured pipl

4.4 Research Gap and Motivation for the Current Project

The reviewed works reveal several important observations. First, strong LLMs can achieve competitive simplification performance through prompt engineering, but direct prompting may not always provide enough control over the type of transformation applied to each sentence. Second, fine-tuning can specialize models for the task, but it may require additional computational resources and does not always guarantee better performance than well-designed prompting. Third, plan-driven systems show that identifying the intended simplification operation before generation can improve the structure and controllability of the output. Fourth, terminology support is particularly important in scientific and medical texts, where difficult terms can reduce readability and lead to misunderstanding.

Based on these observations, the current project is motivated by the need for a controlled sentence-level simplification pipeline that combines operation-aware planning with large language model generation and limited terminology support. The goal is not only to produce simpler sentences, but also to preserve scientific meaning, reduce unnecessary rewriting, and handle different sentence types through suitable simplification strategies.

This direction is supported by the reviewed plan-driven and lexical simplification systems, while also addressing the need for a practical and modular pipeline that can be tested, evaluated, and improved across different sentence categories.

5 Chapter Five: Methodology

5.1 Introduction

This chapter describes the methodology adopted for the design and implementation of the proposed scientific text simplification system. The project addresses sentence-level simplification, where the input is a complex scientific or medical sentence and the output is expected to be a clearer and more accessible version that preserves the original meaning.

The proposed system is based on a controlled large-language-model inference pipeline. The model is not fine-tuned, and its parameters are not updated during the project. Instead, the system controls the model through structured prompts, intermediate decision layers, terminology support, and output validation. This design reflects the nature of scientific and medical simplification, where uncontrolled rewriting may lead to meaning shifts, unsupported additions, excessive deletion, or incorrect simplification of domain-specific terminology.

The methodology is therefore not limited to direct prompting. It decomposes the simplification process into interpretable stages: sentence preparation, terminology extraction, terminology context construction, semantic planning, strategy selection, terminology-aware substitution, strategy-based generation, and evaluation. This structure allows the system to handle different sentence types more carefully and provides traceable intermediate outputs for analysis

5.2 Methodology Overview

The proposed methodology follows a controlled pipeline architecture. Each sentence is processed through a sequence of analytical and generative components. The analytical components identify sentence characteristics such as terminology, structural complexity, statistical density, and evidence-related wording. The generative components use these signals to produce a simplified sentence under explicit constraints.

The general system flow is:

Input complex sentence
→ Text normalization
→ Terminology extraction
→ Terminology context construction
→ Semantic planning
→ Strategy selection
→ Terminology substitution, if applicable
→ Strategy-based generation
→ Output validation
→ Evaluation

This design improves interpretability compared with a direct one-step prompt. A direct prompt would rely entirely on the language model to implicitly decide how to simplify the sentence. In contrast, the proposed pipeline separates analysis from generation. The system stores detected terms, planner output, selected strategy, execution mode, and evaluation results. These intermediate outputs support debugging, error analysis, and comparison between different system variants.

5.3 Dataset and Task Formulation

The project is developed in the context of the CLEF SimpleText sentence-level simplification task. The task requires generating a simplified version of a complex scientific or medical sentence while preserving the original scientific meaning.

The experimental data used in this project is derived from Cochrane-auto sentence-level data. Each example contains a complex sentence, and in the validation and test sets, a reference simplification is available for evaluation. Additional fields such as paragraph identifier and sentence identifier are used when available to retrieve contextual information, especially the true next sentence within the same paragraph.

The task is formulated as an inference problem rather than a supervised training problem. The dataset is used to provide input sentences and reference outputs for evaluation, while the simplification model itself remains a pretrained instruction-following model. This distinction is important because the project contribution lies in designing a controlled simplification pipeline around the model, not in training a new model from scratch.

5.4 Data Preparation and Input Organization

Data preparation ensures that the input examples are represented in a consistent and reliable format before entering the simplification pipeline. Since the system operates at the sentence level, the central input field is the complex scientific sentence. Reference simplifications and labels are preserved when available because they support evaluation and analysis, but they are not used to train the model.

The preparation stage includes verifying the presence of required columns, normalizing textual fields, preserving paragraph and sentence identifiers, and assigning stable row identifiers. The row identifier is particularly important for checkpointing, since it allows the system to track which sentences have already been processed.

The preparation stage also supports controlled experimentation on subsets of the validation or test data. This is useful because LLM-based inference can be computationally expensive, and processing smaller subsets allows iterative analysis of system behavior before scaling to larger runs.

5.5 Model Selection and Inference Setting

The system uses a pretrained instruction-tuned large language model as its main reasoning and generation component. In the current implementation, the model is used locally in inference mode. The model is placed in evaluation mode, and generation is performed without gradient computation. Therefore, the model weights remain unchanged throughout the project.

The same model is used in two different roles. The first role is semantic planning, where the model analyzes the sentence and returns a structured JSON plan. The second role is controlled generation, where the model performs terminology substitution, rephrasing, splitting, or omission according to the selected strategy.

Using the model in these two roles allows the system to benefit from the language understanding ability of the LLM while maintaining explicit control over the simplification process. The model is not treated as a single black-box simplifier; instead, it is integrated into a broader pipeline that constrains and interprets its behavior.

5.6 Terminology Support Component

Scientific and medical terminology is a major source of complexity in the target texts. A general-purpose language model may simplify such terms inconsistently or may choose an unsafe replacement if it is not guided carefully. For this reason, the proposed system includes a terminology-support component.

This component provides glossary-based hints for difficult biomedical terms. It is not a full retrieval-augmented generation system, since it does not retrieve long passages or external documents. Its role is narrower and more controlled: it identifies relevant terms in the input sentence and provides short definitions or safe plain-language equivalents to guide the model during substitution and simplification.

5.6.1 Glossary Loading

The terminology resource is loaded into an internal lookup dictionary. Each entry maps a biomedical term to its corresponding definition. Before storage, terms and definitions are normalized to reduce inconsistencies caused by capitalization, spacing, or formatting differences.

Empty terms and empty definitions are ignored. This prevents incomplete glossary entries from being passed to the model. The resulting dictionary allows efficient lookup during sentence processing.

5.6.2 Filtering and Safe Replacement List

Raw glossary entries cannot be used blindly. Some entries may be too general, too frequent in scientific writing, or unsuitable for substitution. To reduce noisy matches, the system applies filtering rules.

The filtering mechanism includes three main elements. The first is a stop-term list, which excludes general words and common scientific reporting terms that should not trigger terminology substitution. The second is a weak single-term list, which removes isolated words that are too broad to be reliable simplification targets. The third is a curated safe replacement list, which contains medically safer plain-language equivalents for selected terms.

For example, terms such as “hypertension” and “tachycardia” can be guided toward “high blood pressure” and “rapid heart rate.” In contrast, general words such as “chance,” “study,” or “evidence” are not treated as difficult medical terminology in this component.

This filtering design reduces the risk of misleading the model with irrelevant or unsafe terminology hints.

5.6.3 Term Extraction from Sentences

Term extraction identifies which glossary terms actually appear in the input sentence. The system searches multi-word terms before single-word terms. This ordering is important because longer medical expressions may contain shorter terms. Prioritizing longer terms helps preserve complete expressions and avoids extracting partial overlapping terms.

The system also tracks the character spans of detected terms. When a newly detected term overlaps with a previously detected longer term, the new match is ignored. This prevents the same text segment from producing multiple conflicting terminology hints.

The output of this stage is a dictionary containing the detected terms and their definitions or safe replacements. This dictionary represents the terminology evidence available for the current sentence.

5.6.4 Terminology Context Construction

The detected terminology dictionary must be converted into a text format before it can be used by the language model. The system therefore builds a terminology context consisting of short glossary hints.

For example, if the term "glucocorticoids" is detected, the context may include:

glucocorticoids: suggested plain wording = steroid medicines

This context is inserted into the substitution and simplification prompts. Its purpose is to guide lexical simplification while preserving the original medical meaning. The terminology context is not used to introduce new content; it only supports safer rewriting of terms already present in the source sentence.

5.7 Semantic Planning Stage

The semantic planning stage analyzes the sentence before final generation. Its purpose is to transform an unstructured sentence into a structured diagnostic representation that can be used by the strategy selection layer.

The planner returns a fixed JSON schema containing features such as whether the sentence is already simple, whether it contains technical terms, whether it is structurally complex, whether it contains secondary statistical details, whether abbreviation expansion may be required, and whether it depends on the following sentence.

This structured output makes the simplification process more interpretable. Instead of allowing the model to decide implicitly how to simplify the sentence, the planner exposes sentence-level properties that can be inspected and used in later stages.

The planner output is validated before use. If the model returns invalid JSON or omits required keys, the system falls back to a safe default plan. This mechanism prevents a malformed planning response from stopping the entire pipeline.

5.8 Use of Next-Sentence Context

Some scientific sentences are difficult to interpret in isolation. They may refer to information in the following sentence, or they may be redundant with nearby text. To support this analysis, the planner can receive the true next sentence when paragraph and sentence identifiers are available.

The system does not assume that the next row in a sampled dataset is the true next sentence. Instead, it constructs a lookup table using paragraph and sentence identifiers. This ensures that the next-sentence context is retrieved from the original paragraph structure rather than from the sampled row order.

The next sentence is used only as planning context. It helps the planner detect dependency or redundancy, but the final simplification remains based on the original input sentence and the selected strategy.

5.9 Strategy Selection Layer

The strategy selection layer converts the planner output and sentence-level signals into a concrete simplification decision. This layer combines semantic information from the planner with surface-level features such as word count, comma count, clause markers, numerical patterns, evidence wording, and detected terminology.

The system selects one of four main strategies:

KEEP_AS_IS
REPHRASE
SPLIT_AND_SIMPLIFY
DELETE_OR_OMIT

The role of this layer is to prevent the system from applying the same transformation to all sentences. Scientific sentences differ in the type of complexity they contain. A short sentence with a difficult term may need rephrasing, while a long multi-clause sentence may benefit from splitting. A sentence containing secondary statistical details may require compression, but only if it does not contain essential scientific information.

The deletion decision is intentionally conservative. A sentence is selected for omission only when it appears to contain secondary information and does not contain important context such as trials, studies, participants, follow-up, outcomes, effects, evidence, or comparisons. This restriction is necessary because excessive deletion can damage meaning preservation.

The strategy selector is rule-based, but it functions as a control layer around the language model rather than as a replacement for it. The LLM provides semantic planning signals, while the rule-based layer makes the final decision explicit, traceable, and easier to evaluate.

5.10 Execution Stage

The execution stage is responsible for producing the final simplified output. It receives the original sentence, the selected strategy, and the terminology context. Its design ensures that terminology handling is applied before the final sentence-level transformation.

If a real terminology context is available, the system first performs an LLM-based substitution step. This step attempts to replace difficult medical terms with safe plain-language equivalents. If the substitution changes the sentence, the substituted version becomes the working sentence for the final strategy.

The selected strategy is then applied to the working sentence. This ordering is important because it allows the system to clarify difficult terminology before performing rephrasing, splitting, or omission.

5.10.1 Rephrase Strategy

The rephrase strategy is used for sentences that require a clearer wording without major structural transformation. It is suitable for moderately complex sentences, terminology-heavy sentences that are not structurally complex, and cases where no stronger strategy is required.

The prompt emphasizes meaning preservation, avoidance of unsupported information, concise wording, and preservation of uncertainty. This is important in scientific text because uncertain evidence must not be converted into a definite claim.

5.10.2 Split-and-Simplify Strategy

The split-and-simplify strategy targets long or structurally complex sentences. It allows the model to divide a sentence into one or two shorter sentences when splitting improves readability.

This strategy is not intended to force splitting in every case. The model is instructed to split only when it benefits clarity. This prevents unnecessary fragmentation while still supporting simplification of dense scientific sentences.

5.10.3 Delete-or-Omit Strategy

The delete-or-omit strategy is used only for cases where a sentence appears redundant, administrative, or dominated by secondary information that can be safely omitted. The prompt explicitly prevents omission when the sentence contains important scientific information related to studies, participants, outcomes, evidence, or comparisons.

If the sentence contains useful information, the model is instructed to produce a brief simplified version instead of omitting it. This makes the strategy conservative and reduces the risk of losing important content.

5.10.4 Keep-as-is Strategy

The keep-as-is strategy is selected when the sentence is already sufficiently simple and does not require terminology clarification or structural rewriting. This strategy prevents unnecessary changes and helps preserve meaning when simplification is not needed.

If terminology substitution has already been applied before this strategy, the output may correspond to the substituted working sentence. Otherwise, the original sentence is preserved.

5.10.5 Output Validation

Output validation is included to improve system stability. If the selected strategy is keep-as-is, the output is forced to remain the original sentence or the current working sentence. If the model outputs OMIT, the system converts it into an empty output. If the model unexpectedly returns an empty generation for a non-omission case, the system falls back to the input sentence.

This validation step prevents invalid or empty model outputs from being treated as final simplifications.

5.11 Batch Processing and Checkpointing

The system is designed to process multiple sentences through two separate stages: planning and execution. The planner stage saves the sentence, detected terms, terminology context, planner JSON, selected strategy, and processing status into a CSV file. The executor stage reads this file, reconstructs the terminology context, executes the selected strategy, and saves the final output into a separate CSV file.

This separation has two advantages. First, it makes the pipeline easier to inspect because planning and generation results are stored independently. Second, it supports checkpointing. If processing stops unexpectedly, the system can identify completed row IDs and continue from the next unfinished sentence instead of repeating the entire run.

Checkpointing is especially important in LLM-based processing, where generation may require significant time when applied to hundreds or thousands of sentences.

5.12 Evaluation Methodology

The proposed system was evaluated using automatic text simplification and readability metrics. When reference simplifications were available, SARI and BLEU were computed to assess simplification quality and lexical overlap with the reference. In addition, FKGL and compression ratio were used to analyze readability improvement and output length reduction. These metrics were applied to both the validation sample and the test sample, while qualitative examples were inspected to better understand the behavior of the system.

5.13 System Interface

A Gradio interface was developed to demonstrate the behavior of the system on individual examples. The interface allows a user to enter a sentence manually or load an example from the validation data.

The interface displays the original sentence, the generated simplification, detected terminology, terminology handling information, and sentence-level evaluation metrics when a reference is available. Its purpose is to support qualitative inspection of the system rather than replace batch evaluation.

The interface also helps illustrate the difference between direct generation and the proposed controlled pipeline, since it exposes selected diagnostic information such as terminology detection and processing summaries.

5.14 Methodology Summary

This chapter presented the methodology of the proposed scientific text simplification system. The system is designed as a controlled LLM-based inference pipeline rather than a direct prompting baseline. Its main components include terminology extraction, glossary-based context construction, semantic planning, strategy selection, terminology-aware substitution, strategy-based generation, output validation, checkpointing, and automatic evaluation.

The methodology addresses the main challenges of scientific and medical simplification: difficult terminology, long sentence structures, dense statistical information, uncertainty preservation, and meaning faithfulness. By separating analysis from generation, the system provides more control over the simplification process and enables detailed analysis of intermediate decisions and final output.

6 Chapter Six: Results and Discussion

6.1 Introduction

This chapter presents the experimental results of the proposed scientific text simplification system. The evaluation was conducted on two samples of 500 sentences: one from the validation set and one from the test set. The goal of this evaluation is to analyze whether the proposed system can simplify scientific and medical sentences while preserving their original meaning.

The evaluation focuses on the main automatic metrics used in text simplification, including SARI, BLEU, readability level, and compression ratio. In addition, screenshots and selected outputs are included to illustrate the behavior of the system during execution.

6.2 Evaluation Metrics

The proposed system was evaluated using several automatic metrics that capture different aspects of text simplification quality. Since text simplification is not only a generation task but also a meaning-preserving transformation, no single metric is sufficient to evaluate the system completely. Therefore, the evaluation combines simplification quality, lexical similarity, readability, and output length reduction.

SARI was used as the main simplification metric. It evaluates how well the system performs the three main simplification operations: adding appropriate words, keeping important content, and deleting unnecessary content. Unlike metrics that only compare the generated output with the reference, SARI considers the source sentence, the system output, and the reference simplification. This makes it more suitable for text simplification tasks.

BLEU was used as a secondary metric to measure lexical overlap between the generated output and the reference simplification. Although BLEU is widely used in text generation and machine translation, it is not the primary metric in this project because a valid simplified sentence may use different wording from the reference while preserving the same meaning. Therefore, BLEU is interpreted as an additional similarity indicator rather than a complete measure of simplification quality.

FKGL, or Flesch-Kincaid Grade Level, was used to estimate readability. It gives an approximate reading grade level based on sentence length and word complexity. FKGL was computed for both the source sentences and the generated outputs. A lower FKGL score indicates that the sentence is easier to read.

FKGL Delta was calculated as the difference between the FKGL score of the generated output and the FKGL score of the source sentence:

Equation 6.1: FKGL Delta Formula

FKGL Delta = FKGL Prediction - FKGL Source

A negative FKGL Delta indicates that the generated output is easier to read than the original sentence.

Compression Ratio was used to measure output length reduction. It was calculated as:

Equation 6.2 : Compression Ratio Formula

Compression Ratio = Number of words in prediction / Number of words in source sentence

A value below 1 means that the generated sentence is shorter than the original sentence, while a value close to 1 means that the output length is similar to the source sentence. Compression is useful for analyzing conciseness, but it must be interpreted together with meaning preservation because shorter output is not always better if important information is removed.

Together, these metrics provide complementary views of the system behavior. SARI evaluates simplification operations, BLEU measures lexical similarity to the reference, FKGL estimates readability, FKGL Delta measures readability improvement, and compression ratio measures the degree of shortening.

6.3 Results on the Validation Set

Table 2 shows the results obtained on the validation sample.

Table 6.1 : Results on the Validation Sample

Metric	**Result**
SARI	38.98
BLEU	9.26
FKGL Source	12.64
FKGL Prediction	11.33
FKGL Delta	-1.31
Compression Ratio	0.868

The results show that the proposed system achieved a SARI score of 38.98 on the validation set. This indicates that the system was able to perform meaningful simplification operations compared with the reference outputs.

The FKGL score decreased from 12.64 for the source sentences to 11.33 for the generated outputs. This reduction suggests that the simplified sentences became easier to read on average. The negative FKGL delta confirms that the system improved readability.

The average compression ratio was 0.868, which means that the generated outputs were generally shorter than the original sentences. This is useful in scientific text simplification because shorter sentences are often easier for non-specialist readers to understand, as long as the main meaning is preserved.

6.4 Results on the Test Set

Table 6.2 shows the results obtained on the test sample.

Table 6.2 : Results on the Test Sample

Metric	Result
SARI	38.84
BLEU	8.70
FKGL Source	13.39
FKGL Prediction	11.58
FKGL Delta	-1.81
Compression Ratio	0.871

The system achieved a SARI score of 38.84 on the test set, which is close to the validation result. This indicates that the system behavior was relatively stable across the two samples.

The FKGL score decreased from 13.39 to 11.58, giving an average reduction of 1.81 grade levels. This shows that the system improved readability more clearly on the test set than on the validation set.

The compression ratio was 0.871, which is very close to the validation compression ratio. This suggests that the system produced shorter outputs in a consistent way across both datasets.

6.5 Summary of Quantitative Results

Table 4 summarizes the main results on both evaluation samples.

Table 6.3 : Summary of Quantitative Results

Dataset	SARI	BLEU	FKGL Source	FKGL Prediction	FKGL Delta	Compression
Validation	38.98	9.26	12.64	11.33	-1.31	0.868
Test	38.84	8.70	13.39	11.58	-1.81	0.871

The results show that the system achieved very similar SARI scores on the validation and test sets. This stability suggests that the proposed pipeline does not depend only on one specific sample.

The readability results are also positive. In both datasets, the FKGL score decreased after simplification, which means that the generated outputs were easier to read. The compression ratio was also below 1 in both cases, showing that the system generally produced shorter and more concise sentences.

Overall, the quantitative results indicate that the proposed system improves readability and reduces sentence length while maintaining a reasonable simplification performance according to SARI.

6.6 Analysis of Pipeline Design Improvements

During system development, the main improvements were not only related to changing the language model, but also to restructuring the simplification pipeline. Therefore, the reported performance should be interpreted as the result of a controlled pipeline design rather than a simple model replacement.

One important modification was changing the planner output from free-form text into a strict JSON structure. In earlier versions, the planner could return unstructured explanations, which made the decision layer unstable and sometimes caused fallback behavior. After enforcing a fixed JSON schema, the planner output became machine-readable and easier to validate. This made the system more stable because the strategy selector could rely on explicit fields such as whether the sentence contains technical terms, whether it is structurally complex, and whether it contains secondary statistical detail. In addition, diagnostic indicators such as `planner_json_ok` and `used_default` made it easier to detect whether an error came from the model response, the parsing step, or the strategy selection layer.

Another important improvement was separating the planner stage from the executor stage. Earlier versions mixed analysis and generation, making it difficult to identify the source of errors. In the final pipeline, the planner stage saves the sentence analysis, detected terminology, selected strategy, and planner status in `planner_stage.csv`, while the executor stage reads this information and generates the final output in `executor_stage.csv`. This separation improved traceability and supported checkpointing. It also made the system easier to debug because failures in planning could be distinguished from failures in generation.

A third major design change was treating terminology substitution as a preliminary step before the final simplification strategy, rather than as a separate strategy competing with rephrasing, splitting, or omission. In earlier versions, a sentence could be assigned only one strategy, such as `REPHRASE`, `SPLIT`, `DELETE`, or `SUBSTITUTE`. In the final design, when reliable terminology is detected, the system first applies terminology-aware substitution and then applies the selected sentence-level strategy. This allows the system to perform lexical simplification before structural or semantic rewriting. As a result, difficult medical terms can be clarified before the sentence is rephrased or split.

The effect of this change is qualitative as well as quantitative. Outputs became more understandable in terminology-heavy sentences because difficult expressions were more often replaced with safer plain-language wording. For example, terms such as "glucocorticoids," "tachycardia," or "hypotension" can be guided toward clearer expressions such as "steroid medicines," "rapid heart rate," or "low blood pressure." This design also made the execution path more transparent through execution modes such as `rephrase_after_substitution` and `split_after_substitution`.

The strategy selection rules were also refined during development. The decision layer was adjusted to avoid applying the same transformation to all sentences and to reduce overly aggressive splitting or deletion. Numeric and statistical patterns were treated carefully because Cochrane-style sentences often contain important trial counts, confidence intervals, and evidence statements. Therefore, the system was designed to delete or omit information only when it appears secondary and not essential to the scientific meaning.

Overall, these iterations show that the main contribution of the project lies in the design of a controlled LLM-based simplification pipeline. The system is not only a direct prompt to a language model; it includes structured planning, terminology support, strategy selection, staged execution, checkpointing, and diagnostic outputs. For this reason, the final results should be understood as the outcome of the complete pipeline rather than as the effect of a single prompt or a single model.

Table 6.4 : Summary of Pipeline Design Improvements

Component	Earlier Version	Final Version	Effect
Planner Output	Free-form text or unstable response	Strict JSON schema	More stable parsing and clearer decisions
Error Diagnosis	Mixed planning and execution errors	planner_json_ok, used_default, separate stages	Easier debugging
Pipeline Structure	Planning and execution mixed	Separate planner and executor CSV files	Checkpointing and traceability
Terminology Handling	SUBSTITUTE as one possible strategy	Substitution before final strategy	Better lexical simplification
Strategy Selection	Less controlled rules	Refined rules using length, terminology, numeric detail, evidence wording	More controlled transformation choice

6.7 Screenshots of System Results

This section presents screenshots of selected examples processed through the developed system interface. The screenshots show the system behavior on individual scientific and medical sentences, including the input sentence, the generated simplified output, and the displayed system information.

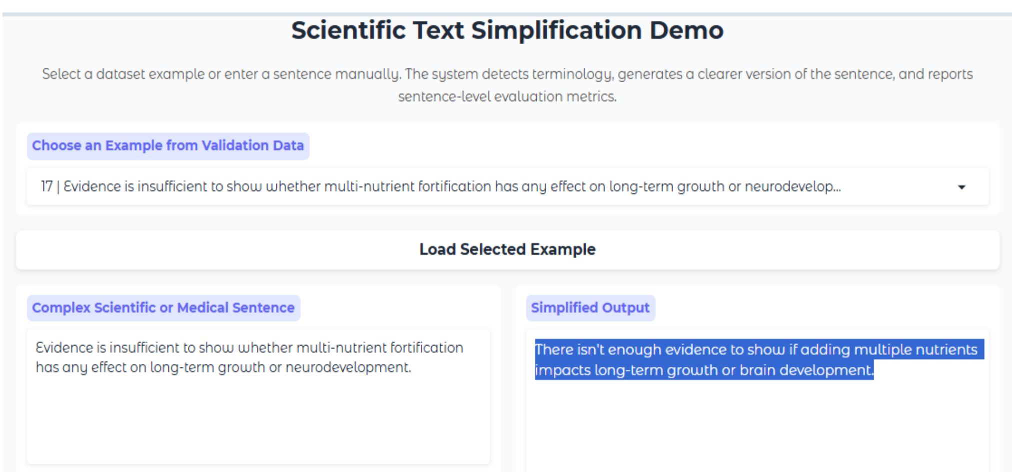

Figure 1 Interface Example 1

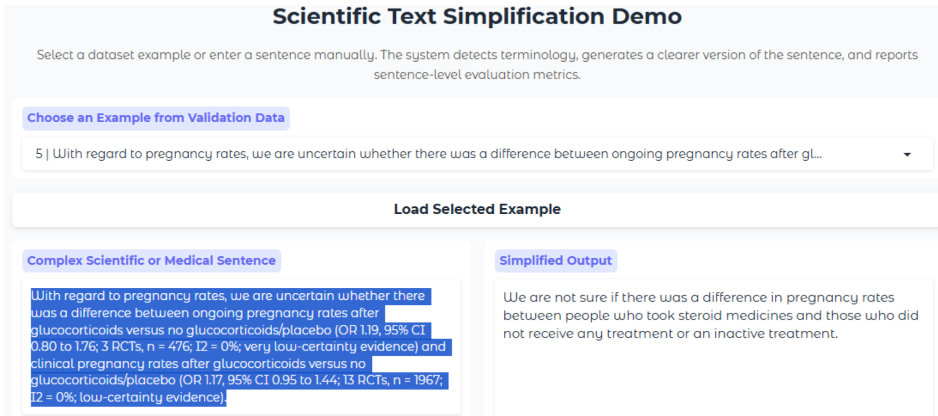

Figure 2 Interface Example 2

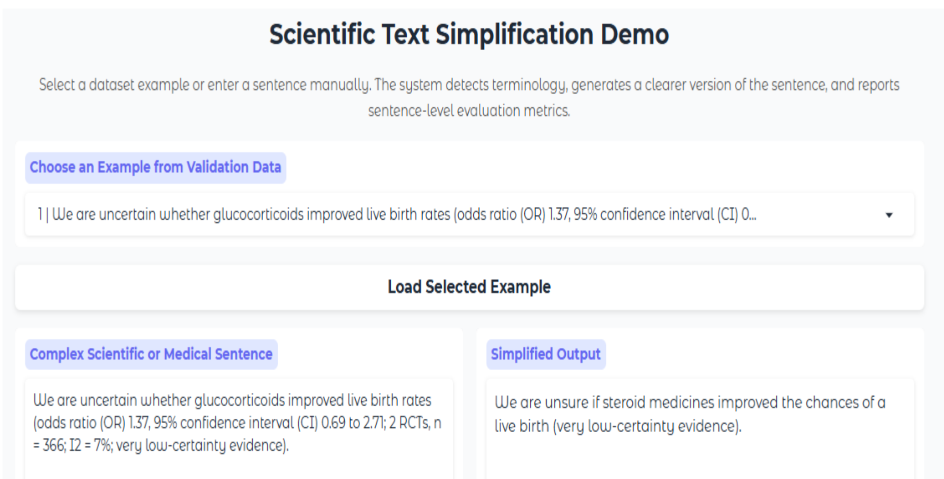

Figure 3 Interface Example 3

.

6.8 Discussion

The results show that the proposed system was able to simplify scientific and medical sentences in a consistent way across the validation and test samples. The SARI scores were close in both experiments, which indicates stable performance.

The decrease in FKGL scores suggests that the generated sentences became easier to read. This is an important result because the main goal of the project is not only to rewrite the text, but also to make it more accessible for non-specialist readers.

The compression ratio also shows that the system generally produced shorter outputs. However, compression alone is not always enough to judge simplification quality. A sentence can be shorter but still inaccurate if important information is removed. Therefore, the system was designed to use planning and strategy selection before generation in order to reduce uncontrolled simplification.

The BLEU scores were relatively low, but this is expected in text simplification tasks. A simplified sentence can be valid even if it does not exactly match the reference sentence. For this reason, SARI and readability measures are more important for evaluating this project.

6.9 Comparison with Related Systems

The proposed system was compared with several systems reported in the CLEF 2025 SimpleText Track. Since the reviewed systems differ in model type, task setting, computational resources, and evaluation conditions, the comparison should be interpreted as an approximate contextual comparison rather than a strict direct ranking.

Several systems achieved higher SARI scores than the proposed system. For example, systems based on strong closed-source models such as GPT-4.1 or GPT-4o achieved competitive results, as shown by SINAI and UBOnlp. Other systems, such as DS@GT, used plan-guided simplification and achieved a higher sentence-level SARI score. Similarly, terminology-supported approaches such as LIS showed strong performance, especially when external glossary resources were used effectively.

Compared with these systems, the proposed system achieved a lower SARI score than the strongest reported systems, but its performance remained within a reasonable range for sentence-level scientific text simplification. The validation and test results of the proposed system were close to each other, which suggests stable behavior across samples. In addition, the system achieved a clear reduction in FKGL and produced shorter outputs on average, indicating improved readability and conciseness.

The proposed system is especially close in direction to plan-driven and terminology-supported approaches. Similar to plan-guided systems, it does not rely on a single direct prompt, but analyzes the sentence before generation and selects a suitable simplification strategy. Similar to terminology-supported systems, it uses glossary-based hints to support medical term simplification. However, the current system differs by using a controlled local inference pipeline with structured JSON planning, rule-based strategy selection, terminology-aware substitution, and checkpointed batch execution.

It is also important to note that the goal of the current project was not only to maximize SARI, but also to build an interpretable and controllable simplification pipeline. This is relevant in scientific and medical text simplification, where preserving meaning and avoiding unsupported changes are important. Therefore, even though some previous systems achieved higher numerical scores, the proposed system contributes a practical pipeline design that combines planning, terminology support, and strategy-based execution.

Some reviewed systems reported results close to or below the proposed system. For example, certain fine-tuned or encoder-decoder systems achieved SARI scores in the mid-to-high 30s. This indicates that the proposed pipeline is competitive with some supervised or smaller-model approaches, despite not fine-tuning the model. At the same time, the gap between the proposed system and the strongest LLM-based systems suggests that further improvements are still possible, especially through stronger models, larger-scale evaluation, better terminology resources, and more systematic ablation studies.

Overall, the comparison shows that the proposed system does not outperform the strongest CLEF 2025 systems, but it achieves stable and reasonable results while following a controlled and interpretable methodology. The results support the usefulness of combining semantic planning, terminology support, and strategy-based simplification, and they provide a foundation for future development.

6.10 Chapter Summary

This chapter presented the experimental results of the proposed scientific text simplification system. The evaluation was conducted on validation and test samples using SARI, BLEU, FKGL, and compression ratio.

The results showed that the system achieved close SARI scores on both samples, reduced the FKGL readability level, and produced shorter outputs on average. These findings indicate that the proposed system can improve the readability of scientific and medical sentences while maintaining stable simplification behavior.

The comparison with previous systems and related work will be discussed in a later section after completing the final evaluation and organizing the comparative results.

7 References

[1] J. Bakker, B. Vendeville, L. Ermakova, and J. Kamps, “Overview of the CLEF 2025 SimpleText Task 1: Simplify Scientific Text,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4167–4185.

[2] B. Vendeville, J. Bakker, H. Azarbonyad, L. Ermakova, and J. Kamps, “Overview of the CLEF 2025 SimpleText Task 2: Identify and Avoid Hallucination,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4186–4204.

[3] M. M. Agüero-Torales, C. Rodríguez Abellán, and C. A. Castaño Moraga, “Sentence-level Scientific Text Simplification With Just a Pinch of Data,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4205–4210.

[4] G. Arampatzis and A. Arampatzis, “DUTH at CLEF 2025 SimpleText Track: Tackling Scientific Text Simplification and Hallucination Detection,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4211–4224.

[5] J. Collado-Montañez, J. A. Ortiz-Zambrano, C. Espin-Riofrio, and A. Montejo-Ráez, “SINAI in SimpleText CLEF 2025: Simplifying Biomedical Scientific Texts and Identifying Hallucinations Using GPT-4.1 and Pattern Detection,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4225–4239.

[6] A. A. Dongre, A. Vaadiraaju, and A. K. Madasamy, “NITK SCaLAR Lab at the CLEF 2025 SimpleText Track: Transformer-Based Models for Biomedical Sentence Simplification (Task 1.1),” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4240–4254.

[7] S. Eugin, B. A, S. V, and S. V, “RECAIDSTechTitans at CLEF 2025: Simplifying Scientific Text and Identifying Spurious Sentences using T5,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4255–4262.

[8] Y. Gallina, T. Jiménez, and S. Huet, “University of Avignon at the CLEF 2025 SimpleText Track: Guided Medical Abstract Simplification,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4263–4275.

[9] N. Hofmann, J. Dauenhauer, N. O. Dietzler, I. D. Idahor, and C. K. Kreutz, “THM@SimpleText 2025 - Task 1.1: Revisiting Text Simplification based on Complex Terms for Non-Experts,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4276–4285.

[10] S. Huet and E. Sanjuan, “A Benchmark Collection for Assessing Scholarly Search by Non-Educated Users,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4286–4300.

[11] P. Kocbek and G. Stiglic, “UM_FHS at the CLEF 2025 SimpleText Track: Comparing No-Context and Fine-Tune Approaches for GPT-4.1 Models in Sentence and Document-Level Text Simplification,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4301–4310.

[12] N. Largey, D. Wu, and B. Mansouri, “AIIRLab Systems for CLEF 2025 SimpleText: Cross-Encoders to Avoid Spurious Generation,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4311–4323.

[13] K. C. Marturi and H. H. Elwazzan, “Hallucination Detection and Mitigation in Scientific Text Simplification using Ensemble Approaches: DS@GT at CLEF 2025 SimpleText,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4324–4337.

[14] K. C. Marturi and H. H. Elwazzan, “LLM-Guided Planning and Summary-Based Scientific Text Simplification: DS@GT at CLEF 2025 SimpleText,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4338–4347.

[15] A. A. N. Djoudi, S. Nouali, M. Aabid, I. Badache, A.-G. Chifu, and P. Bellot, “LIS at SimpleText 2025: Enhancing Scientific Text Accessibility with LLMs and Retrieval-Augmented Generation,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4348–4355.

[16] T. Papandreou, J. Bakker, and J. Kamps, “University of Amsterdam at the CLEF 2025 SimpleText Track,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4356–4362.

[17] B. Vendeville, L. Ermakova, P. De Loor, and J. Kamps, “UBOnlp Report at the SimpleText lab of CLEF 2025,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4363–4375.

[18] A. Vora, T. Chaudhari, S. Hotha, and S. Sonawane, “S-3 Pipeline for Biomedical Text Simplification,” in Working Notes of CLEF 2025: Conference and Labs of the Evaluation Forum, CEUR Workshop Proceedings, vol. 4038, 2025, pp. 4376–4387.

[19] CLEF 2026 SimpleText Track, "CLEF 2026 SimpleText Track: Home," 2026. [Online]. Available: https://simpletext-project.com/. [Accessed: May 13, 2026].

[21] Codabench, "CLEF 2026 SimpleText Task 1," 2026. [Online]. Available: https://www.codabench.org/competitions/16108/. [Accessed: May 13, 2026].